Physics-Aware Reinforcement Learning for Topology Synthesis of Microfluidic Flow Distributors

Parv Goyal
Computer Science and Engineering
Indian Institute of Technology Guwahati
Guwahati, India
g.parv@iitg.ac.in

Sukanta Bhattacharjee
Computer Science and Engineering
Indian Institute of Technology Guwahati
Guwahati, India
sukantab@iitg.ac.in

***Abstract*—Microfluidic flow distributors split one inlet stream into several outlets with prescribed, often non-uniform, flow ratios. Recent automated design tools, notably the μFluidicGenius framework, still require the user to draw the channel topology in addition to the target flow rates. We remove this requirement: the user specifies only the target outlet flow rates, and the topology is synthesised automatically. The design space is the sequential placement of tiles from a library of pre-characterised serpentine-channel variants on an 8×8 grid. A hydraulic circuit solver evaluates each candidate in about a millisecond, and its flow predictions agree with two-dimensional COMSOL simulations to about 13%. A proximal policy optimisation (PPO) agent acts under an action mask that guarantees every episode ends in a closed, leak-free topology. A multi-objective reward couples flow accuracy with compactness and inlet pressure, and a progress- and time-gated curriculum, a top-K design archive and a fixed evaluation protocol make the results reproducible. An automated pipeline then assembles the synthesised chip geometry and checks its outlet flows in COMSOL. On representative targets the solver-predicted outlet flows are within the specification. We close by describing BTP-2, which realises the synthesised topology with resistance-tuned maze channels, following μFluidicGenius.**

***Index Terms*—Microfluidics, flow distribution, reinforcement learning, topology synthesis, hydraulic circuit model, proximal policy optimisation, COMSOL Multiphysics.**

I. Introduction

Microfluidic flow distributors deliver a single inlet stream to several outlets in prescribed proportions. They are building blocks of organ-on-chip platforms, chemical micro-reactors and screening devices, where each chamber must receive a defined perfusion rate. The classical design route computes channel resistances with the electric-circuit analogy [2] and then trims channel lengths by trial and error. Continuum topology optimisation with Navier–Stokes solvers avoids this but is expensive and yields free-form boundaries that are hard to fabricate. Assembling a chip from pre-characterised modular tiles guarantees manufacturability, but creates a large discrete design space.

Motivation from μFluidicGenius. Birtek *et al.* [1] recently reported μFluidicGenius (μFG), an open tool that lets a non-expert place reservoirs and channels on a grid and assign flow rates. μFG builds a circuit matrix by the electrical analogy and solves a mixed-integer linear program (MILP) for the channel resistances. It then realises every resistance as a maze channel generated by tabu search, guided by an instance-based ML regressor trained on 328,320 COMSOL simulations. Three 3D-printed circuits with two, three and four outlets reproduced the target flow distributions with about 90% accuracy. In μFG, however, *the topology is an input*. The topology fixes which resistances are required, the inlet pressure and the footprint, so a hand-drawn layout need not be a good one, especially for asymmetric splits. This observation motivates the present work: *the user supplies only the target flow rates, and the topology itself is generated by a learning agent*, with the aim of obtaining layouts that are at least as compact and low-pressure as hand-drawn ones.

Learning-based design automation has been explored for droplet generators [3], for randomly generated micro-fluidic layouts [4] and for interactive layout tools, and reinforcement learning has been applied to chip floorplanning [5]. To our knowledge, none of these synthesises a hydraulically closed flow-distribution topology from target flow rates alone.

Contributions.

1) A modular tile library (50 side assignments, 640 variants) with COMSOL-extracted leg resistances, and a 290-node nodal circuit solver verified against two-dimensional finite-element simulations.
2) A tile-placement MDP whose action mask, built on a closure oracle, *guarantees* that every episode terminates in a topology with no open ports.
3) A structure-aware reward and training protocol (batch advantage normalisation, progress- and time-gated curriculum, top-K archive, fixed evaluation), including the failure modes we diagnosed along the way.
4) An automated post-synthesis COMSOL validation pipeline that rebuilds the exact tile geometry of a synthesised chip.
5) A concrete plan (BTP-2) to realise the synthesised topologies following μFG.

II. Hydraulic Model and Tile Library

A. Circuit analogy

We consider square channels of width and depth $100\,\mu$m carrying water ($\mu = 1.01 \times 10^{-3}\,$Pa s, $\rho = 998.2\,$kg/m^3) at $U_0 = 6\,\mu$l/min. The Reynolds number is $\mathrm{Re} = \rho U_0 D_h/\mu \ll 1$, so the flow is in the Stokes regime, $\Delta P = R\,Q$, and Kirchhoff's

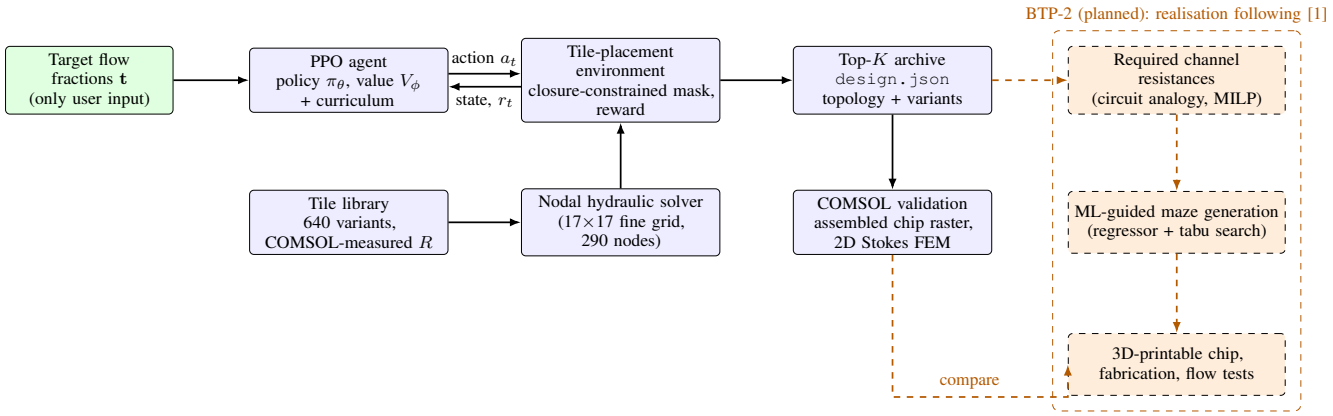


Fig. 1. Overall framework. Solid blocks are completed in this report (BTP-1). Dashed blocks are planned for BTP-2: given the synthesised topology and the target flow rates, resistances are assigned to channels and realised as maze channels, as in μFG [1].

laws apply. For a rectangular duct of width w, depth $h \leq w$ and length L,

$$R = \frac{12\,\mu L}{w h^3\,(1 - 0.63\,h/w)}, \tag{1}$$

which gives $r_0 \approx 3.28 \times 10^8\,\mathrm{Pa\,s\,m^{-3}}$ per μm of channel for the $100\,\mu$m square duct.

B. Nodal Circuit Solver

The 8×8 coarse grid of tile cells is mapped onto a 17×17 fine grid of nodes (289 total nodes, plus one virtual inlet node 289, giving 290 nodes). As shown in Fig. 2, each cell (r, c) has:

- A central node at coordinates $(2r + 1, 2c + 1)$.
- Four boundary port nodes at $(2r, 2c + 1)$ (North), $(2r + 2, 2c+1)$ (South), $(2r+1, 2c)$ (West), and $(2r+1, 2c+2)$ (East).

Because adjacent cells share the exact same boundary port node, pressure and flow continuity across tile interfaces are satisfied automatically by construction.

Every microchannel leg with resistance R_{ij} connects node i to node j with hydraulic conductance $g_{ij} = 1/R_{ij}$. Under laminar Stokes flow, the flow rate Q_{ij} between nodes i and j follows Ohm's fluidic law:

$$Q_{ij} = g_{ij}(P_i - P_j). \tag{2}$$

Applying conservation of mass (Kirchhoff's Current Law, $\sum Q_{\text{out}} = Q_{\text{in}}$) at each node yields three simple cases:

1) **Internal Nodes**: Total fluid entering must equal total fluid leaving:

$$\sum_{j \in \mathcal{N}(i)} g_{ij}(P_i - P_j) = 0 \tag{3}$$

where $\mathcal{N}(i)$ is the set of neighbors connected to node i.

2) **Outlet Port Nodes** ($k \in \mathcal{O}$): Fluid leaves through an exterior stub resistor R_{stub} into the ambient atmosphere ($P_{\text{amb}} = 0$):

$$\sum_{j \in \mathcal{N}(k)} g_{kj}(P_k - P_j) + g_{\text{stub}} P_k = 0. \tag{4}$$

3) **Virtual Inlet Node** ($i = \text{in}$): A constant volumetric flow Q_{in} is injected into the inlet port:

$$g_{\text{stub}}(P_{\text{in}} - P_{\text{inlet_port}}) = Q_{\text{in}}. \tag{5}$$

Combining these equations for all connected nodes forms the linear system:

$$\mathbf{AP} = \mathbf{b} \tag{6}$$

where $\mathbf{P}$ is the vector of unknown nodal pressures, the matrix $\mathbf{A}$ contains conductances:

$$A_{ii} = \sum_{j \in \mathcal{N}(i)} g_{ij}, \quad A_{ij} = -g_{ij}, \tag{7}$$

and the source vector $\mathbf{b}$ has Q_{in} at the inlet node and 0 elsewhere.

Since $\mathbf{A}$ is symmetric and positive-definite, solving Eq. (6) via Cholesky decomposition takes under 1 ms. The resulting discharge flow at each outlet k is simply:

$$Q_k = g_{\text{stub}} P_k = \frac{P_k}{R_{\text{stub}}}. \tag{8}$$

For parallel branches stemming from a shared junction, this recovers the standard current divider rule: each outlet receives flow inversely proportional to its branch resistance ($Q_k \propto 1/R_k$).

If an active tile port does not connect to an adjacent tile, it becomes a dead end. For that isolated port node, Eq. (3) simplifies to $g(P_{\text{port}} - P_{\text{centre}}) = 0$, forcing $P_{\text{port}} = P_{\text{centre}}$ and zero flow. Consequently, the linear solver treats unconnected open ports as sealed walls and cannot penalize physical fluid leaks on its own. This directly motivates the topological closure reward mechanism introduced in Section III-B.

C. Tile library

A tile has a non-empty inlet-side set I and a disjoint non-empty outlet-side set O drawn from $\{N, S, E, W\}$. This gives 50 side assignments and, for each, several variants that differ in the meander depth of every leg (Table I). A leg is a self-avoiding path on a 13×13 grid-line lattice from the centre junction to the face midpoint. A meander level $w \in \{0, \ldots, 5\}$

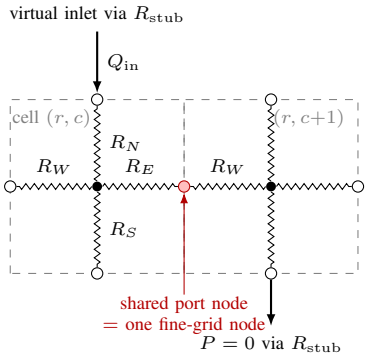


Fig. 2. Nodal model. Each tile is a star of leg resistances around a centre node. Adjacent tiles share the port node on their common edge.

TABLE I
COMPOSITION OF THE TILE LIBRARY.

Type (in–out)	Side assignments	Variants each	Variants
1–1	12	6	72
1–2	12	12	144
1–3	4	20	80
2–1	12	12	144
2–2	6	20	120
3–1	4	20	80
Total	50		640

folds the leg inside its own quadrant, so legs never intersect, and a level-w leg is $1 + w$ times as long as a straight one. Rasterised at 100 μm per pixel, a tile is a 25×25 binary mask (2.5 mm edge, Fig. 3).

Leg resistances are extracted with COMSOL (two-dimensional laminar flow with a 100 μm out-of-plane thickness). One port is driven at 6 μl/min and all others are sunk at $p = 0$. Only the driven pressure p_{in} and the sink flows q_k are observable, so the junction pressure is estimated by assuming resistance proportional to leg length:

$$p_c = p_{\text{in}} \frac{l_{\text{eq}}}{l_d + l_{\text{eq}}}, \quad R_d = \frac{p_{\text{in}} - p_c}{q_{\text{in}}}, \quad R_k = \frac{p_c}{q_k}, \tag{9}$$

where l_d is the driven leg length and $l_{\text{eq}} = (\sum_k 1/l_k)^{-1}$ the parallel combination of the sink legs. A unit-less baseline library with $R \propto$ leg length is also provided for machines without COMSOL.

III. Topology Synthesis by Reinforcement Learning

A. Problem and MDP

The chip has one inlet at cell $(0, 3)$ (north face) and $J = 3$ outlets at $(7, 0)$, $(7, 3)$, $(7, 7)$ (south faces) on an 8×8 grid. Given target flow fractions t_j with $\sum_j t_j = 1$ (so $Q_j^* = t_j Q_{\text{in}}$), we seek a tile placement of at most $n_{\max} = 25$ tiles such that every outlet lies within its target. An episode places one tile per step. An action is a pair (cell, library variant) among 64×640 choices, plus a STOP action. The policy therefore chooses topology and resistances jointly.

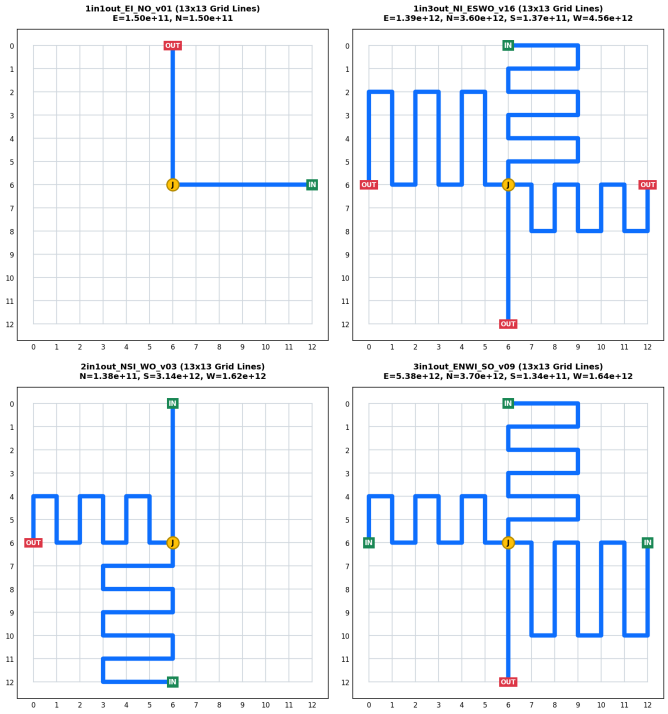
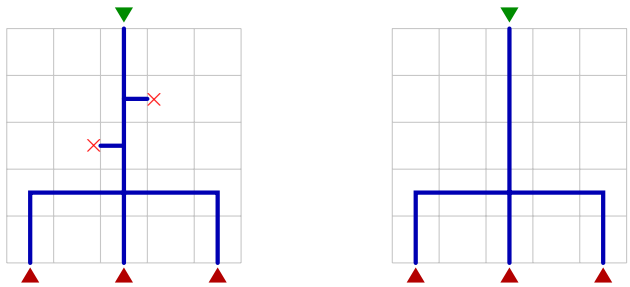


Fig. 3. Examples of library tiles (13×13 grid-line view): straight and meandered 1–1 tiles, a 1–3 and a 2–1 splitter and a 3–1 merge tile. Green: inlet, red: outlet; J marks the centre junction.



(a) with open ends (b) closed

Fig. 4. Closure on a 5×5 illustration. (a) Two legs end in empty cells (red crosses); the solver sees zero flow in them. (b) The closed topology has no open port.

B. Closed topologies by construction

For every active side of a placed tile, the neighbour across it is either (a) the chip boundary at the inlet or outlet port, (b) a placed tile whose facing side has the opposite role, (c) an *empty cell* (a *pending* port), or (d) anything else, which is illegal. We call a topology *closed* when no pending, open-inlet or blocked port remains and all outlet cells are placed (Fig. 4). Pending ports are unavoidable during growth, but the final design must have none.

The action mask enforces closure with four rules.

1) *Exact-inlet rule.* A tile placed at an empty cell must have as its inlet set exactly the sides whose neighbours already export into the cell (the implicit north side for the inlet cell). Outlet sides may only face empty in-bounds cells.

Algorithm 1 Closure-constrained action mask

Require: board B, spec, allowed side combinations $\mathcal{A}$, budget $n_{\max}$

```
rep ← AnalyzePorts(B) {demand map D, pending, missing outlets}
if rep is closed then
    return mask = {Stop}
end if
for all empty cell X ∈ D do
    I ← D[X] {exact-inlet rule}
    for all admissible O ⊆ free sides of X with (I, O) ∈ A do
        B' ← B ∪ {X : (I, O)}
        if Oracle.Plan(B') ≠ ∅ within n_max − |B'| then
            enable every library variant of (I, O) at X
        end if
    end for
end for
return mask
```

Chip-outlet cells accept only a south-only outlet. A new tile never exports into a placed tile, so the flow graph is a directed acyclic graph.

2) *STOP only when closed.*
3) *Viability filter.* A tile action is allowed only if a bounded depth-first *closure oracle* finds a completion that closes all pending ports within the remaining budget $n_{\max} - |B|$. The oracle replays the real placement rules, prunes with necessary conditions (pending demand and unplaced outlets must coexist, the number of needed cells fits the budget, Manhattan lower bounds, and every demanded cell must reach an unplaced outlet through empty cells), caches results by board signature, and treats a node-limit hit as "not viable".
4) *Budget reserve.* The oracle plan length counts against $n_{\max}$.

C. Reward

Let $E_Q = \frac{1}{J}\sum_j \min\big(|Q_j - Q_j^*|/\max(Q_j^*, 0.01Q_{\text{in}}),\, 3\big)$ be the clipped mean relative flow error. The core cost is $C = \lambda_q E_Q + \lambda_c\, n/n_{\max} + \lambda_v u$, where n is the tile count and u the number of unconnected outlets. The dense step reward is

$$r_t = C_{t-1} - C_t + \lambda_{\text{conn}}\Delta\kappa + \lambda_{\text{bal}}\Delta\beta + \lambda_\phi\Delta\phi \qquad (10)$$

with κ the fraction of connected outlets, $\beta = 1 - \frac{1}{2}\sum_j |\alpha_j - \beta_j|$ the similarity of the realised and target splits, and ϕ a potential equal to the negative normalised mean Manhattan distance from placed tiles to unreached outlets. At termination the agent additionally receives $+1$ if all outlets are within ε (success), $+0.3$ if connected with $0.05 < E_Q \le 0.15$, $-\lambda_{\text{iso}} n_{\text{iso}}$ for isolated tiles.

Because the dense terms are differences of state functions, they telescope, so the return equals terminal quality plus constants. Structure therefore enters only through the terminal structural penalty

$$S = \lambda_x\, x + \lambda_\ell\, \ell, \qquad (11)$$

with normalised terms $x = \text{clip}\big(\frac{n - n_{\min}}{n_{\max} - n_{\min}}, 0, 1\big)$ (extra tiles), $\ell = \min(\text{loops}, 8)/8$, where loops is the cycle rank $E - V + C$ of the tile graph. The presets are `legacy` $(0, 0)$, `compact` $(0.30, 0)$, `low-pressure` $(0.15, 0)$ and `tree` $(0.30, 0.10)$ for $(\lambda_x, \lambda_\ell)$. The sum of structural weights is capped at 0.6 so that success always outranks a cheap failure. The terms are added once at episode end, so the dense reward is unchanged.

D. Policy, training and curriculum

Each valid action is described by a 39-dimensional feature vector: 18 global features (connected outlets, outlet flow fractions, relative errors, E_Q, fill ratio, ϕ, per-branch resistance and resistance deficit), 8 cell, 7 variant and 3 progress features, 2 alignment/diversity features and a STOP flag. A shared MLP $f_\theta(x_a) = w_2^\top \tanh(W_1 x_a + b_1) + b_2$ scores every valid action independently, and $\pi(a|s)$ is the softmax over valid actions. This gives a variable-size action set without a fixed output head. A small resistance-compatibility prior (scale 0.5) is added to the logits: it favours low-resistance variants when the nearest outlet has a flow deficit and high-resistance ones otherwise. A value network V_ϕ on the global features supports generalised advantage estimation (GAE). Training uses PPO [6] with the clipped surrogate and an entropy bonus. Advantages are normalised over the whole update batch rather than per episode, because per-episode centring removes the difference between good and bad episodes. Hyper-parameters are in Table II. Everything is implemented in NumPy with manual back-propagation and Adam.

Actions are masked [7]. A curriculum [8] controls the start state: *warmup* pre-places the inlet tile and the tile below it, *easy* pre-places only the inlet tile, and *normal* starts from an empty board (an optional fourth stage tightens the tolerance). An episode counts as good if it is a success or is closed with $E_Q \le 0.15$. The stage advances when 40% of the last 50 episodes are good (after at least 100 episodes), when a stage has used 25% of the budget, or, unconditionally, so that at least 40% of all episodes run at the normal stage.

E. Selection and evaluation

The reported design is not the best of the stochastic training episodes. Instead, every closed successful episode updates a top-K archive ($K = 5$) holding at most one design per topology hash and ordered by $(-\text{score}, n, p_{\text{in}}, \text{design id})$, where $\text{score} = R - S$. After training, 200 stochastic episodes and one greedy episode are run from an empty board with a separate random stream and summarised by success rate, tile count, pressure ratio, loop count and number of distinct topologies.

IV. Verification and Validation Pipeline

A. Solver versus COMSOL

Before using the solver as the reward model, it was benchmarked against two-dimensional COMSOL laminar-flow

TABLE II
MAIN SETTINGS OF THE SYNTHESIS RUNS.

Parameter	Value
Grid / max tiles	8×8 / 25
PPO: γ, GAE λ, clip, epochs, mini-batch	0.99, 0.95, 0.2, 4, 64
Hidden units (policy / value)	128 / 64
Episodes / rollouts per update	2000 / 8 (16 in batch runs)
Closure oracle node limit	4000

simulations. Random connected 8×8 layouts joining the inlet to all three outlets were generated by constrained random walks with leg lengths. Each layout was solved by the nodal solver and by a COMSOL model built automatically through the Java API ($p = 0$ at the outlets). The outlet flows were compared by their L_1 and L_∞ differences and by the relative error $L_1 / \sum_j Q_j$.

B. Post-synthesis validation of a final design

The solver check uses synthetic layouts with equal channel properties. A synthesised chip instead combines tiles with *different* meander depths. Our validation harness therefore rebuilds the exact geometry of every placed tile from the library recipe, using the same 25×25 rasters whose resistances were measured, and stacks them on the grid into a 200×200-pixel chip mask. Adjacent tiles join pixel to pixel at their port pixels. The harness then (i) checks the assembled mask (no lost pixels, tile-to-tile contacts only where legs face each other, ports open); (ii) recomputes the solver flows from the stored per-leg resistances; and (iii) solves the chip in COMSOL through the existing runner (inlet velocity at the north port, $p = 0$ at the south ports) and reports the outlet flow fractions against the targets and the solver.

V. RESULTS AND DISCUSSION

A. Solver accuracy and cost

Over the benchmark chips the nodal solver and COMSOL agree to within a mean flow error of about 13%. The small residual is consistent with bend and contraction losses that the one-dimensional model omits. Mass is conserved to machine precision in the solver ($< 10^{-14}\,\mathrm{m}^3/\mathrm{s}$). A solver evaluation takes about a millisecond, whereas one COMSOL run takes 45–90 s, a difference of four to five orders of magnitude ($> 50{,}000\times$ speedup). This cost ratio is what makes closed-loop reinforcement learning with thousands of evaluations feasible. Quantitative error metrics aggregated across benchmark runs are reported in Table III.

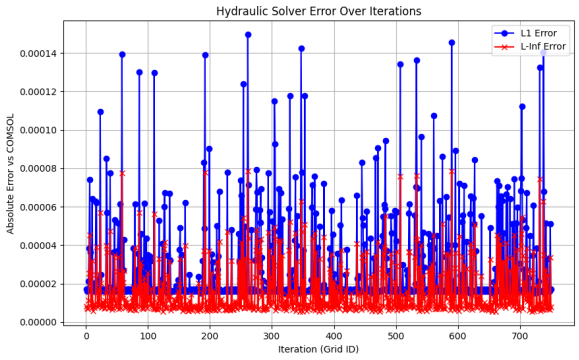
B. Tile library results

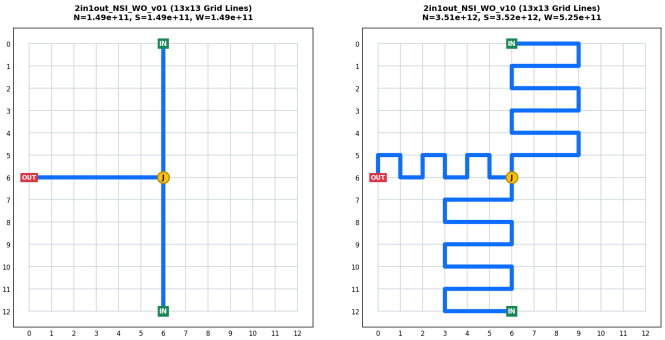
The discrete modular library abstracts complex microchannel segments into pre-characterized canonical tiles within a standardized footprint ($W = H = 100\,\mu\mathrm{m}$). Each tile maps active fluidic ports (N, S, E, W) to calibrated hydrodynamic resistances based on analytical Poiseuille flow ($r_0 \approx 3.28 \times 10^8\,\mathrm{Pa \cdot s/m^3 \cdot \mu m}$).

Figure 5 illustrates the two primary classes of library elements: (a) basic topological connectors (straight channels, 90° turns, T-junction splitters, and cross-junctions) providing low baseline resistance ($R \approx 3.93 \times 10^{11}\,\mathrm{Pa \cdot s/m^3}$), and (b) integrated serpentine delay channels that pack extended hydraulic lengths ($L \in [1200, 4800]\,\mu\mathrm{m}$) within the fixed tile boundaries. These serpentine variants allow the agent to tune branch resistances over an order of magnitude without requiring auxiliary off-chip extensions.

TABLE III
AGGREGATE ERROR AND COMPUTATIONAL PERFORMANCE OF THE HYDRAULIC CIRCUIT SOLVER EVALUATED AGAINST COMSOL NAVIER–STOKES SIMULATIONS.

Metric	Absolute Error	Percentage Error
Mean L_∞ Error	0.1364	13.64%
Mean L_∞ Error(Adjusted)	0.1344	13.44%



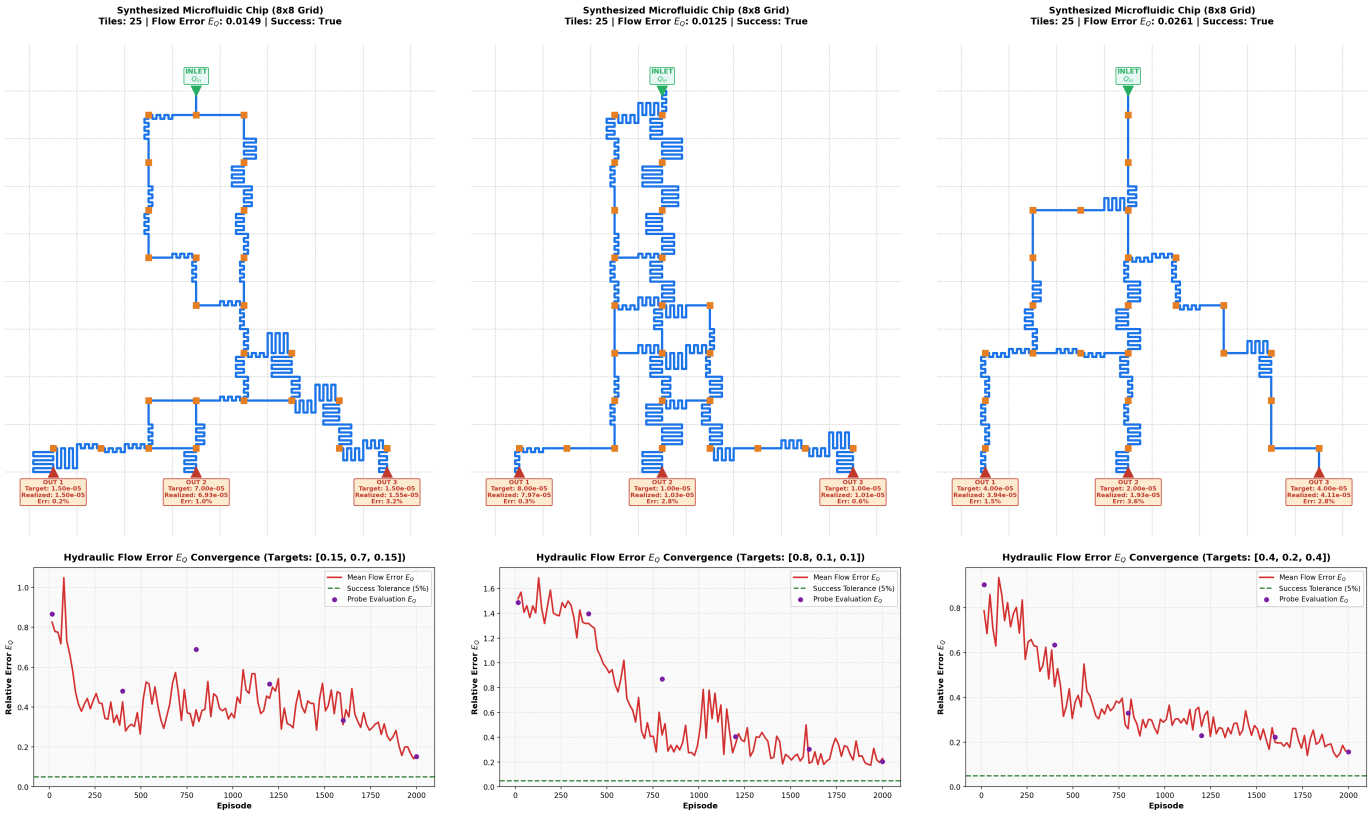


(a) Primitive tile geometries (b) Serpentine resistance mazes

Fig. 5. Characterized microfluidic tile library: (a) primitive routing connectors defining cardinal port connectivity (N, S, E, W), and (b) parameterized serpentine delay mazes providing high hydrodynamic resistances within a compact unit cell.

C. Reinforcement learning synthesis results

The reinforcement learning agent was evaluated across diverse target flow specifications over 2000 episodes using the three-stage curriculum. The policy converges stably and the closure penalty completely eliminates open microchannel ends. Table IV summarizes the quantitative performance across representative target distribution. Subsequent cross-validation in COMSOL Multiphysics demonstrates close agreement with continuous 2D Navier-Stokes simulations.



(a) Target: 0.15 / 0.70 / 0.15 (b) Target: 0.80 / 0.10 / 0.10 (c) Target: 0.40 / 0.20 / 0.40

Fig. 6. Synthesised microfluidic flow distributor topologies generated by the reinforcement learning framework for three representative target flow splits along with Flow error learning curve across 2000 episodes of training.

TABLE IV
SUMMARY OF RL SYNTHESIZED FLOW DISTRIBUTOR DESIGNS ACROSS TARGET FLOW SPLITS.

Target Split	Solver	Realised Flows	Max Error (%)
0.33 / 0.34 / 0.33	0.3262 0.3376 0.3363	0.3694 0.3428 0.2877	4.86
0.20 / 0.30 / 0.50	0.2006 0.2935 0.5060	0.1388 0.2238 0.6374	13.14
0.40 / 0.40 / 0.20	0.3993 0.3937 0.2071	0.3445 0.3438 0.2173	5.48
0.45 / 0.10 / 0.45	0.4532 0.0971 0.4497	0.3768 0.0733 0.5499	10.02
0.55 / 0.25 / 0.20	0.5668 0.2422 0.1910	0.4484 0.3548 0.1968	11.84
0.80 / 0.10 / 0.10	0.7967 0.1028 0.1005	0.7900 0.1218 0.0882	1.90

D. Limitations

(i) The geometry is fixed to one inlet, three bottom outlets and an 8×8 grid. (ii) Tile resistances are quantised to 640 variants, so extreme targets may require extra tiles. (iii) The tile model assumes length-proportional junction pressure (Eq. (9)) and ignores bend and junction coupling, and the COMSOL model is two-dimensional with an imposed thickness. (iv) Nothing has been fabricated yet, so manufacturing effects such as dimensional deviation of printed channels are not captured.

VI. SCOPE OF BTP-2

The topology synthesis completed here provides exactly what μFG [1] takes as user input: a channel topology together with target flow rates. BTP-2 will therefore follow the μFG methodology, with the RL-generated topology in place of the user-drawn one (Fig. 1):

1) **Required resistances.** From the synthesised topology, the target flow rates and an inlet pressure, build the circuit matrix by the electrical analogy and compute the resistance of every channel segment. As in μFG, the system has more unknowns than equations because intermediate pressures are free. We will resolve this with a MILP that favours low-complexity solutions, and extend it to topologies with merges and loops, which μFG does not consider.
2) **Maze generation.** Realise each required resistance with a maze channel found by tabu search over path modifications, using the regressor as the fitness evaluator

and approaching the target from below to allow for fabrication-induced resistance increase [9].

3) **Export and fabrication.** Export printable geometry (STL or mould) including pre-designed junctions and ports, respecting the minimum feature size of the printer, and fabricate the chips.
4) **Experimental validation and comparison.** Measure outlet flow rates with flow sensors, compare them with the RL solver and with COMSOL, and compare RL-generated topologies with hand-drawn ones on footprint, inlet pressure and accuracy. We target at least the $\sim$90% accuracy reported in [1].
5) **Closing the loop.** Feed fabrication tolerances back into the reward through the robustness term ρ, and relax the fixed tile library by letting the RL stage select among continuous maze resistances.

VII. Conclusion

We presented a physics-aware reinforcement-learning framework that synthesises modular microfluidic flow-distributor topologies from target flow rates alone. A verified nodal circuit solver provides millisecond evaluations, and a closure-constrained action mask guarantees that every generated chip is free of open ports. A structure-aware reward, batch-normalised PPO, a progress- and time-gated curriculum, a top-K archive and an automated COMSOL validation harness complete the pipeline. The diagnosed failure modes (invisible open ends, budget filling, a stalled curriculum and best-of-N selection bias) show that the reward and the evaluation protocol matter as much as the learning algorithm. BTP-2 will realise the synthesised topologies with resistance-tuned mazes following μFluidicGenius and validate them experimentally. The code is open source GitHub.

References

[1] M. T. Birtek, V. Aktas, B. Aktas, A. C. Abdullah, A. Ozcan, and S. Tasoglu, "ML-automated microfluidic circuit design," *Sci. Adv.*, vol. 12, no. 5, eaea7598, Jan. 2026.

[2] K. W. Oh, K. Lee, B. Ahn, and E. P. Furlani, "Design of pressure-driven microfluidic networks using electric circuit analogy," *Lab Chip*, vol. 12, pp. 515–545, 2012.

[3] A. Lashkaripour *et al.*, "Machine learning enables design automation of microfluidic flow-focusing droplet generation," *Nat. Commun.*, vol. 12, no. 25, 2021.

[4] J. Wang, P. Brisk, and W. H. Grover, "Random design of microfluidics," *Lab Chip*, vol. 16, pp. 4212–4219, 2016.

[5] A. Mirhoseini *et al.*, "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021.

[6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.

[7] S. Huang and S. Ontanón, "A closer look at invalid action masking in policy gradient algorithms," in *Proc. FLAIRS*, 2022.

[8] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, "Curriculum learning," in *Proc. ICML*, 2009, pp. 41–48.

[9] F. Glover, "Tabu search – Part I," *ORSA J. Comput.*, vol. 1, no. 3, pp. 190–206, 1989.